

IEEC YA CONNECT - ARCHITECTURE HANDBOOK

Platform Design Authority and Engineering Standards

Field | Value
Document Owner | IEEC YA IT Team
Version | 0.4
Status | Draft - Architecture Baseline (Chapters 1-13)
Classification | Internal Use
Technology Stack | React (web) + React Native / Expo (mobile) + Firebase
Last Updated | July 13, 2026
Prepared by | IEEC YA IT Team
Architecture and AI Design Support | OpenAI ChatGPT / Cursor

Revision History

Version | Date | Status | Changes | Owner
0.1 | July 12, 2026 | Draft | Chapter 1 added | IEEC YA IT Team
0.2 | July 12, 2026 | Draft | Chapter 2 added; handbook consolidated | IEEC YA IT Team
0.3 | July 13, 2026 | Draft | Chapter 3 added; multi-organization and federated parent model approved; Person and calendar scope clarified | IEEC YA IT Team
0.4 | July 13, 2026 | Draft | Chapters 4-13 added (People, AuthN, AuthZ, Forms, Workflow, Calendar, Chat, Notifications/Tasks, Audit, Data & Engineering); web + React Native mobile clients settled | IEEC YA IT Team

Table of Contents

- 1. Chapter 1 - Purpose, Scope, and Design Authority
 - 2. Chapter 2 - Core Design Principles and Shared Platform Engines
 - 3. Chapter 3 - Organization and Tenant Model
 - 4. Chapter 4 - People and Account Model
 - 5. Chapter 5 - Identity and Authentication
 - 6. Chapter 6 - Authorization and Permission Engine
 - 7. Chapter 7 - Dynamic Forms Engine
 - 8. Chapter 8 - Workflow Engine
 - 9. Chapter 9 - Ministry Calendar Engine
 - 10. Chapter 10 - Chat and Collaboration
 - 11. Chapter 11 - Notifications and Tasks
 - 12. Chapter 12 - Audit and History
 - 13. Chapter 13 - Data and Engineering Standards
-

Chapter 1 - Purpose, Scope, and Design Authority

1.1 Purpose

This handbook defines the mandatory platform-wide architecture rules for IEEC YA Connect. Every module, feature, developer, and AI coding assistant must follow these rules unless an approved Architecture Decision Record changes them.

1.2 Platform Vision

IEEC YA Connect is a centralized ministry management platform for managing people and ministry operations. Its primary mission is to help leaders shepherd people through the ministry journey while maintaining clear accountability, secure access, and reliable history.

Newcomer -> Member -> Minister -> Leadership

1.3 Scope

- People and account management
- Ministry teams, positions, and oversight
- Follow-up and attendance
- Calendar, communication, and collaboration
- Permissions and approvals
- Dynamic forms and workflows
- Notifications, tasks, reports, and audit history
- Support for multiple independent organizations and optional parent organizations

1.4 Design Authority

The handbook is the platform design authority. If a module specification conflicts with this handbook, the handbook takes precedence unless an approved Architecture Decision Record explicitly changes the standard.

1.5 Mandatory Foundation Rules

- One permanent Person record per individual within each organization.
- Each organization owns and isolates its people, users, ministries, teams, groups, calendar, chat, files, reports, roles, and permissions.
- Roles are permission templates, not final authority.
- Default deny and least privilege.
- Technical authority is separate from ministry authority.
- Oversight is separate from team membership.
- One shared operational calendar per organization.
- Parent-level events may affect selected organizations only through explicit scope and policy.
- Chat membership is separate from team membership.
- Forms and workflows are configurable where practical.
- Soft delete by default.
- Important actions and changes are audited.
- React UI restrictions never replace backend enforcement.

Chapter 2 - Core Design Principles and Shared Platform Engines

2.1 Purpose

This chapter defines the reusable engines and engineering principles that every ministry module must use. Modules must extend these shared services rather than building isolated copies.

2.2 People-Centered Architecture

Within an organization, the Person record is the center of the ministry data model. Newcomer journeys, membership status, team assignments, attendance, reports, chats, calendar participation, and permissions reference the same organization-owned Person identity. A status change must never create a duplicate Person within the same organization.

2.3 Configuration Before Hard-Coding

Ministry processes change over time. Forms, field options, approval steps, deadlines, reminder rules, sensitivity categories, status labels, and role defaults should be configurable whenever that can be done safely and clearly. Configuration must not weaken security or allow invalid workflows.

2.4 Permission Engine

The Permission Engine determines who may view or perform each action. Roles provide default permissions, while individual overrides can explicitly grant or deny permissions. Explicit deny takes precedence over grants. Permission changes must be scope-aware, time-aware when configured, and fully audited.

- Default deny
- Role-based default grants
- Individual grant and deny overrides
- Organization, ministry, team, group, workflow, and record scopes
- Oversight-scoped permissions
- Time-based assignments
- Backend enforcement
- Permission source traceability

2.5 Workflow Engine

The Workflow Engine manages approvals, state transitions, required steps, and exceptions. It must support configurable one-step or multi-step approval, sequential or parallel steps, return-for-correction, rejection, cancellation, and complete transition history.

2.6 Dynamic Form Engine

The Dynamic Form Engine supports configurable forms for newcomer reports, welcome schedules, member profiles, applications, event registration, and future ministry processes. Published forms are versioned so previous submissions always retain the exact structure used at submission time.

- Short text and long text
- Yes/no
- Single-select and multi-select
- Date and time
- Number
- Required or optional fields
- Conditional fields
- Configurable labels and order
- Versioned publication
- Permission-controlled form management

2.7 Ministry Calendar Engine

Each organization uses one shared operational calendar. Ministries, teams, and groups create events inside that calendar according to permission and conflict rules. Parent organizations may create parent-level events that are published to selected organizations and may block local scheduling only when the event scope and conflict policy explicitly require it.

2.8 Chat and Collaboration Engine

Teams may have multiple chat channels. Chat membership is independent of team membership, allowing selected ministers from other teams to join a channel without receiving team permissions or access to protected ministry records. Chat data remains organization-owned and isolated.

2.9 Notification and Task Engine

Notifications and tasks are shared platform services. They support in-app notifications, push notifications, email, and future SMS or WhatsApp integrations. The system must distinguish informational notices from required actions and must track delivery, read status, due dates, and escalation.

2.10 Audit Engine

The Audit Engine records important actions, including permission changes, assignments, report submissions, attendance corrections, status changes, workflow decisions, chat membership changes, calendar overrides, and parent-level oversight access. Audit records should be append-only and protected from normal client modification.

2.11 Shared Supporting Services

- File and photo storage
- Search and duplicate detection
- Organization configuration
- Profile completion
- Approval templates
- Sensitive-data classification
- Soft deletion and restoration
- Reporting and analytics

2.12 Enforcement Layers

- React UI: shows only permitted screens and actions.
- Application service layer: validates workflows and business rules.
- Firestore Security Rules: prevents unauthorized direct reads and writes.
- Cloud Functions or trusted backend: performs privileged operations and audit logging.
- Automated tests: verify both allowed and denied behavior.

2.13 Architecture Decision Records

Major architecture choices must be recorded as ADRs. An ADR includes the decision, context, alternatives considered, consequences, status, and approval date. ADRs explain why the platform was designed a certain way and help future developers avoid reversing critical decisions accidentally.

2.14 Initial ADR Catalog

ADR Decision Status

ADR-001 One organization-owned Person
record per individual

Approved

ADR-002 Roles are permission templates Approved

ADR-003 One shared operational calendar
per organization

Approved

ADR-004 Chat membership independent
of team membership

Approved

ADR-005 Dynamic forms as a shared
platform service

Approved

ADR-006 Soft delete and audit by default Approved

ADR-007 Federated multi-organization
tenant model

Approved

2.15 Completion Criteria

A future module is architecture-compliant only when it reuses the shared engines, defines its permissions and data scope, protects sensitive data, preserves history, supports audit, and avoids duplicating person, calendar, chat, workflow, or form infrastructure.

Chapter 3 - Organization and Tenant Model

3.1 Purpose

This chapter defines how IEEC YA Connect hosts multiple organizations, how organizations may be grouped under a parent organization, who owns operational data, and how access is isolated. These rules apply to every collection, permission check, query, file path, background job, report, calendar event, and chat channel.

3.2 Approved Tenant Model

IEEC YA Connect will use a federated multi-organization model. Each local organization is an independent tenant that owns and controls its operational data. Multiple local organizations may be connected to an optional parent organization for denomination-wide coordination, standards, aggregate reporting, and explicitly granted oversight.

IEEC YA Connect Platform

?-- Parent Organization (optional)

 ?-- Local Organization A

 ? ?-- Ministries

 ? ? ?-- Teams

 ? ? ? ?-- Groups

 ? ? ?-- Shared ministry services

 ? ?-- Organization-owned people, users, calendar, chat, files, and reports

 ?-- Local Organization B

 ?-- Independently owned and isolated operational data

3.3 Core Terms

Platform: The IEEC YA Connect software and shared technical infrastructure. Platform administration concerns software operation, not automatic ministry-content access.

Parent Organization: An optional denomination, network, or governing body that groups multiple local organizations. It receives only explicitly configured oversight and aggregate access.

Organization: An independent church, branch, congregation, or institutional tenant. It owns its people, users, ministries, teams, groups, calendar, chat, files, reports, roles, permissions, and configuration.

Ministry: A major organizational area inside one organization, such as Young Adult, Women, Children, Youth, or Worship Ministry.

Team: An operational unit inside a ministry, such as Follow-Up, Bible Study, Media, Sound, Worship, or Usher.

Group: A subgroup inside a team or ministry, such as a G5 group, Bible Study group, class, cohort, or temporary working group.

3.4 Organization Independence

Each organization operates independently even when it belongs to a parent organization. It determines its own ministry structure, people records, status definitions, role templates, permission assignments, forms, workflows, chat channels, calendar events, reports, and local configuration within platform safeguards.

- Organization A cannot read or modify Organization B data by default.
- A local administrator has authority only within assigned organization scope.
- Parent affiliation does not remove local ownership.
- Platform technical roles do not automatically receive pastoral or ministry-content access.
- No local role or permission crosses into another organization unless a separate assignment explicitly grants it.

3.5 Organization-Owned People

Each organization owns its own Person records. The same human may exist in more than one organization, but each organization maintains a separate profile, ministry status, history, roles, permissions, attendance, notes, and assignments. Records are not automatically merged or shared across organizations.

Same human

?-- Washington Organization Person Profile

? ?-- Status: Minister

? ?-- Team: Follow-Up

? ?-- Local history and permissions

?-- Maryland Organization Person Profile

?-- Status: Member

?-- Team: None

?-- Separate local history and permissions

A verified sign-in identity may later be associated with access to more than one organization, but this does not combine the organizations' Person records or ministry data. The authentication and organization-access model will be finalized in Chapter 5.

3.6 Data Ownership and Required Scope

Every organization-owned operational record must carry an organization scope. Records belonging to a ministry, team, group, workflow, or specific case must also include the relevant lower-level scope identifiers.

- organizationId - required for organization-owned operational data
- parentOrganizationId - used only when relevant to a parent relationship or parent-owned record
- ministryId - required when the record belongs to a ministry
- teamId - required when the record belongs to a team
- groupId - required when the record belongs to a group
- scopeType and scopeId - used for generic permission, configuration, workflow, and reporting services

A document must never rely only on a client-selected path or UI context to determine its organization.

Security rules and trusted backend operations must verify the scope from authoritative records.

3.7 Parent Organization Oversight

A parent organization may coordinate multiple local organizations, but it does not automatically own or receive unrestricted access to local operational data. Parent access is permission-based, purpose-limited, and auditable.

3.7.1 Typical Parent-Level Capabilities

- View the list and status of affiliated organizations
- View approved aggregate statistics
- Publish denomination-wide announcements
- Create parent-level programs and calendar events
- Maintain shared policies, templates, or standards
- Receive selected reports submitted by local organizations
- Assign specifically scoped oversight responsibilities

3.7.2 Restricted by Default

- Pastoral and sensitive bio notes
- Individual newcomer follow-up reports
- Private local chat channels
- Local files and attachments
- Local permission details
- Individual financial records

- Any person-level data not required for an approved parent function

3.8 Scope Types

The permission and configuration engines must understand these scopes:

Scope Purpose Example

platform Technical operation of the software platform

Create or suspend organization tenants

parent_organization Denomination or network coordination

View approved aggregate reports

organization One local church or tenant Manage local ministries and people

ministry One ministry inside an organization

Manage Young Adult ministry configuration

team One operational team Assign Follow-Up newcomers

group One subgroup Manage one Bible Study group

workflow One workflow instance or workflow type

Approve a membership recommendation

record One specific protected record View one restricted escalation

3.9 Organization Structure

The default IEEC structure is:

Parent Organization -> Organization -> Ministry -> Team -> Group

The platform may support configurable labels and optional levels in the future, but the initial implementation should preserve these explicit concepts because they carry different business and security meanings. A Group is not the same as a Team, and a Parent Organization is not the same as a local Organization.

3.10 Local Calendar and Parent Events

Each organization has one shared operational calendar. All local ministries, teams, and groups create events inside that calendar according to permission and conflict rules. A parent organization may maintain parent-owned events and publish them to selected organizations.

- Parent events must declare affected organizations.
- Parent events must declare whether they are informational, warning-only, approval-required, or hardblocking.
- A parent event must not silently delete an existing local event.
- When a new parent event creates a conflict, affected local event owners must be notified and the conflict must enter a review process.
- Parent calendar overrides and exceptions must be audited.

3.11 Chat and Collaboration Boundaries

Local organization chats remain local and private by default. Parent organizations may create parent-level channels for selected participants from affiliated organizations. Joining a parent channel does not grant access to the participant's local organization data, and joining a local chat does not grant team membership or module permissions.

3.12 Roles and Permissions Placement

Chapter 3 defines where authority applies. Chapter 6 will define how roles, permissions, individual overrides, oversight assignments, and explicit denials are resolved. Every assignment must include a scope and must never be interpreted beyond that scope.

Person: Daniel

Role template: Follow-Up Leader

Scope: Washington Organization / Young Adult Ministry / Follow-Up Team

Result: Follow-Up management defaults only inside that team, subject to overrides

3.13 Organization Lifecycle

Organizations must support a controlled lifecycle without deleting operational history.

- draft - tenant setup is incomplete
- active - normal access and operation
- suspended - user access or selected services are temporarily blocked
- archived - organization is no longer active but retained for history and legal requirements
- deleted - exceptional permanent deletion process, restricted and separately governed

Suspension by a platform administrator must not silently transfer ownership or expose local ministry content. The system should allow controlled export, restoration, and retention processes.

3.14 Cross-Organization Operations

Cross-organization functions must be explicit shared workflows rather than direct access. Examples include a parent announcement, a denomination-wide event, an aggregate report submission, a shared training registration, or an approved transfer process. Each workflow must define what data is sent, who may receive it, and whether local approval is required.

3.15 Security Requirements

- Every request must resolve the active organization context from trusted account membership or assignment data.
- Firestore queries and document reads must verify organization scope.
- Cloud Storage paths and access checks must include organization ownership.
- Background jobs must process records only within declared scope.
- Parent aggregate reports should prefer precomputed or intentionally submitted data instead of unrestricted access to local raw records.
- All parent oversight access and cross-organization data movement must be audited.
- Organization IDs supplied by the client must never be trusted without authorization checks.

3.16 ADR-007 - Federated Multi-Organization Tenant Model

Status Approved

Decision IEEC YA Connect supports multiple independent organizations grouped under an optional parent organization.

Local ownership Each organization owns and isolates its people, users, roles, permissions, ministry records, calendar, chat, files, configuration, and operational history.

Parent access Parent organizations receive only explicitly configured oversight, aggregate reporting, shared standards, announcements, and event capabilities.

People The same human may have separate organization-owned Person profiles. Profiles are not automatically shared or merged.

Security consequence Every operational record and permission assignment must be organization-scoped, and cross-organization access is denied by default.

Reason This supports denomination-wide coordination without sacrificing local ownership, privacy, or independent ministry operations.

3.17 Chapter Completion Criteria

- The platform recognizes Platform, Parent Organization, Organization, Ministry, Team, and Group as distinct scopes.
- Every local operational record belongs to one organization.
- Organization-owned people and users are isolated from other organizations.
- Parent access is explicit, limited, and audited.
- Each organization uses one shared operational calendar.
- Parent events can affect selected organizations only through declared policies.
- Roles and permissions never cross scope automatically.
- Cross-organization workflows transfer only approved data.

Chapter 4 - People and Account Model

4.1 Purpose

This chapter defines how IEEC YA Connect represents people, ministry progression, user accounts, invitations, duplicates, profile completion, and multi-organization identity. Every module that stores or displays a person must follow these rules.

4.2 Core rule - Person is the center

Within an organization, the Person record is the permanent center of the ministry data model.

- Newcomer journeys, membership, team membership, attendance, reports, chats, calendar participation, roles, and permissions all reference an organization-owned Person.
- A status change must never create a duplicate Person inside the same organization (ADR-001).
- Modules must not invent parallel "person-like" stores. They may hold operational records that point to personId.

4.3 Person != User Account

Concept | Meaning

Person | Organization-owned ministry identity and profile

User Account | Authentication identity that can sign in to the platform

Rules:

1. A Person may exist without a User Account (public newcomer registration).
2. A User Account must link to exactly one Person per organization context it serves (see Chapter 5 for multi-org sign-in).
3. Creating a Person does not automatically create login credentials.
4. Creating a User Account does not create a second Person; it links to an existing Person.

Firebase Auth UID maps to userAccounts/{authUid} -> personId. Person exists first whenever possible.

4.4 Organization ownership of Person profiles

Each organization owns and isolates its Person records (ADR-007).

- The same human may have separate Person profiles in different organizations.
- Profiles are not automatically shared or merged across organizations.
- Cross-organization workflows may transfer only approved fields under explicit policy; they do not merge Person graphs by default.

Human: Daniel

?-- IEEC YA (Washington) Person - newcomer history, YA roles, local attendance

?-- Maryland Org Person - separate profile, status, and history

4.5 Separated identity concepts (do not collapse)

These remain distinct fields / assignment types. Do not store them as one "role" string on Person.

Concept | Purpose | Examples

Ministry Status | Journey stage in the organization | Newcomer, Member, Minister

System Role | Technical platform authority | Super Admin, System Admin, Support Admin

Organizational Position | Ministry-wide leadership position | Head Leader, Core Team

Team Role | Scoped operational authority via RBAC templates | Follow-Up Leader, Assistant Leader, Minister

Team Membership | Belonging to a team (ops access baseline) | Follow-Up team member

Oversight Assignment | Can oversee selected teams without being operational member | Oversight of Media + Usher

Volunteer | Temporary assignment; not a ministry status | Event volunteer (optionally time-bound)

Ministry status answers "where is this person in the shepherding journey?"

Roles and positions answer "what may this person do?"

They must stay independent so a Member can lead a team and a Minister can temporarily have no login.

4.6 Ministry status model

4.6.1 Canonical statuses

Primary progression:

Newcomer -> Member -> Minister

Additional record / lifecycle flags (separate from ministry status when needed):

- recordStatus: e.g. active, inactive, archived (soft lifecycle of the Person record)
- Journey state (Follow-Up): operational workflow state on the journey document, not a second Person identity

4.6.2 Status history

Every ministry status change writes history:

- Previous status, new status
- Reason / workflow reference (e.g. membership approval id)
- Actor (changedBy), timestamp
- Optional notes

Never overwrite history in place. Soft-delete or supersede incorrect entries with audited corrections (ADR-006).

4.6.3 Status change rules

- Promoting Newcomer -> Member happens through approved membership workflow, not by elapsed time alone.
- Attendance and weekly contact inform readiness but are never the only factor.
- Transition to Member updates Person currentMinistryStatus, keeps prior status in history, and completes the Follow-Up journey per module rules.
- Do not create a new Person when status changes.

4.7 Progressive profile completion

Person profiles are incomplete by design at first contact.

1. Public registration / first capture - minimum fields needed for Follow-Up (name, contact, sex, contact preference, etc. per form config).
2. Operational enrichment - ministers and leaders add bio notes, corrected contacts, photos, and ministry fields over time.
3. Member / Minister enrichment - additional profile sections may unlock as status and permissions allow.

Rules:

- Missing optional fields must not block shepherding work.
- Required fields are form/config driven (Dynamic Forms engine), not hard-coded everywhere.
- Sensitive profile fields require explicit permissions to view/edit.
- Profile edits are audited when they change identity-critical or sensitive data.

4.8 Returning people

When someone returns after inactivity or a prior journey:

1. Search the same organization Person store first (normalized name, phone, email).
2. Prefer reactivating / continuing the existing Person over creating a new one.
3. A new Follow-Up journey may be created on the same Person; ministry status rules determine whether they remain Newcomer or other status.
4. Prior history (attendance, reports, bio, previous journeys) remains attached to the Person.

4.9 Duplicate detection and merge

4.9.1 Detection

Public registration and leader-created people run duplicate candidates against organization-scoped Person data using normalized fields (name, phone, email). Possible matches place the journey/submission in `duplicate_review_required` (or equivalent) until resolved.

4.9.2 Review outcomes (audited)

Authorized reviewers may:

- Link submission to existing Person
- Create a new Person (false match)
- Update contact fields on existing Person
- Mark submission as duplicate / discard
- Escalate

4.9.3 No auto-merge

The system must not auto-merge Person records. Merge is a deliberate, permissioned, audited operation that:

- Chooses a surviving Person
- Re-points operational references (personId) safely
- Soft-deletes or archives the duplicate Person
- Preserves history of both records

Cross-organization merge is out of scope for default behavior (ADR-007).

4.10 User accounts and invitations (People side)

Account lifecycle details live in Chapter 5. People-model rules:

Person type | Account expectation

Newcomer | Normally no account

Member | Normally receives an invitation after membership approval (not auto-generated passwords at approval time unless policy explicitly says otherwise)

Minister / people with system responsibilities | Require an account when they must operate in the system

Leaders / Admins | Require an account

Invitation is an intentional access grant linked to an existing Person. Leaders may delay Member invitations when ministry process requires it; the default path is invite-after-approval.

4.11 Soft delete of people

Soft delete by default (ADR-006):

- Person recordStatus moves to inactive/archived rather than hard delete
- Operational history remains for audit and pastoral continuity
- Hard delete is exceptional, highly privileged, and normally out of MVP scope

4.12 Relationships and entities (People Engine)

The People Engine owns (or coordinates) at least:

- people - Person profiles
- userAccounts - account ? person link (Auth chapter)
- ministryStatusHistory
- teamMemberships
- organizationalPositionAssignments
- oversightAssignments
- volunteerAssignments (or equivalent time-bound membership)
- Relationships to other people/entities as modules need (e.g. household links later)

Follow-Up journeys, reports, attendance, and bio are module records that reference personId; they are not stored as nested forever-growing arrays inside Person.

4.13 Multi-organization access (People view)

A signed-in human may access more than one organization only when:

1. Each organization has (or creates under policy) its own Person profile for that human, and
2. A User Account / org-membership link authorizes that organization context (Chapter 5).

Selecting an active organization switches which Person profile and permissions apply. Data from org A is not readable in org B by default.

4.14 Security and privacy consequences

- Person PII is organization-scoped.
- Public registration must not expose internal admin fields.
- Contact and sensitive bio visibility is permission-gated.
- UI hiding is not security; backend rules enforce Person access (Chapters 5-6).

4.15 Chapter completion criteria

- Person is organization-owned and permanent within that org (ADR-001).
- Person != User Account is enforced in model and workflows.
- Ministry status, system roles, org positions, team roles, oversight, and volunteers remain separated.
- Status changes write history and never spawn duplicate Persons.
- Progressive profiles, returning-person handling, and duplicate review (no auto-merge) are defined.
- Member invitations default to post-approval; ministers with responsibilities require accounts.
- Soft delete and audit apply to Person lifecycle and merges.
- Multi-org humans use separate Person profiles per organization.

4.16 Next chapter

Chapter 5 defines Identity and Authentication: Firebase Auth relationship, invitation/activation, account statuses,

multi-org sign-in context, and security boundaries between Auth and ministry status.

Chapter 5 - Identity and Authentication

5.1 Purpose

This chapter defines how humans authenticate into IEEC YA Connect, how Auth identities link to organization-owned Person records, how invitations and activation work, and how multi-organization sign-in context is selected. Authentication proves who signed in. It does not by itself grant ministry authority (Chapter 6).

5.2 Separation of concerns

Layer | Responsibility

Firebase Auth | Credentials, session, email verification, password reset

User Account (userAccounts/{authUid}) | Platform account record linking Auth UID -> Person(+ org context)

Person | Ministry identity and profile (Chapter 4)

Permission Engine | What the signed-in Person may do (Chapter 6)

Rules:

- A valid Auth session without a linked User Account / Person must not receive operational permissions.
- Ministry status (Newcomer/Member/Minister) is not an Auth claim and must not be treated as a login role.
- Disabling an account must not delete the Person.

5.3 Identity chain (canonical)

Firebase Auth UID

```
-> userAccounts/{authUid}
  -> personId (+ organization membership / context)
    -> roleAssignments, overrides, teamMemberships, ...
      -> effective permissions
```

MVP assumption for a single-org deployment: userAccounts documents include organizationId and personId for the primary organization. Multi-org deployments may add explicit organizationMemberships (or equivalent) while keeping one Auth UID per human login identity.

5.4 Who needs an account

Situation | Account required?

Public newcomer registration | No

Assigned Follow-Up minister recording reports/attendance | Yes

Follow-Up Leader / Assistant with system duties | Yes

Core Team / Head Leader approving membership | Yes

System / Support Admin | Yes

Member browsing member features | Yes (after invitation / activation)

Read-only public pages (registration, public info) | No

Default Member path: invitation after membership approval (Chapter 4). Do not auto-create passwords silently unless a future ADR/policy explicitly enables that mode.

5.5 Invitation and activation lifecycle

5.5.1 States (account)

Recommended accountStatus / invitation fields:

State | Meaning

invited | Invitation issued; Auth user may or may not exist yet

pending_activation | Auth exists; waiting for password set / email verify / first login

active | May sign in and use granted permissions

disabled | Sign-in blocked or session rejected for app use

revoked | Invitation/account intentionally withdrawn

Track at least: invitationStatus, invitedAt, invitedBy, activatedAt, lastLoginAt, emailVerified.

5.5.2 Flow

1. Authorized leader/admin selects Person -> Invite
2. System records invitation against Person / User Account stub
3. Invitee receives email (or approved channel) with secure activation link
4. Invitee sets credentials via Firebase Auth
5. userAccounts/{authUid} linked to personId + organizationId
6. Person.hasUserAccount = true
7. accountStatus -> active (after required verification steps)
8. Audit: invitation, activation, and link events

5.5.3 Rules

- Invitation always targets an existing Person (create Person first if needed).
- One active User Account link per Person per organization (no duplicate active logins for the same Person in that org without an audited remediation path).
- Re-invite is allowed when prior invite expired or failed; prior attempts remain in audit/history.
- Activation must not invent permissions; permissions come only from Chapter 6 assignments.

5.6 Authentication methods (baseline)

- Email / password via Firebase Auth for web and mobile.
- Password reset and email verification use Firebase-supported flows on both clients.
- Mobile sessions use the same Auth UID -> User Account -> Person chain as web.
- Additional providers (Google, SSO, phone, biometrics as device UX) are future options and must still resolve to the same User Account -> Person chain.

5.7 Session and active organization context

On each app session:

1. Validate Firebase Auth session.
2. Load userAccounts/{uid}.
3. Resolve allowed organization(s).
4. Establish active organization context (single-org MVP: the only org).
5. Load that org's Person profile for the account.
6. Resolve effective permissions for that Person in that org (Chapter 6).
7. Deny access if account is disabled/revoked or Person is archived without break-glass admin policy.

Switching organizations (when enabled) reloads Person + permissions for the selected org. Never mix org A permissions with org B data.

5.8 Account status vs ministry status

These must never be collapsed:

Field | Lives on | Examples

Ministry status | Person | newcomer, member, minister

Account status | User Account | invited, active, disabled

Record status | Person | active, archived

Journey state | Follow-Up journey doc | assigned, membership_review, ...

Examples:

- Person is Member, account still invited -> cannot use member app features until activation.
- Person is Newcomer, no account -> Follow-Up continues via assigned ministers' accounts.
- Person is Minister, account disabled -> Person record remains; login blocked; reassignment of work may be required.

5.9 Multi-organization sign-in

Under ADR-007:

- One human login (Auth UID) may be authorized for multiple organizations.
- Each organization still has its own Person profile.
- Cross-organization access is denied by default.
- Parent-organization oversight uses explicit parent-scoped permissions, not ambient access to child Person stores.

Implementation detail (planning): prefer explicit membership documents over packing unbounded org lists into Auth custom claims alone. Claims may cache org ids for rules performance but Firestore remains source of truth.

5.10 Security requirements

1. Backend enforcement - Firestore Security Rules (and Cloud Functions where used) verify Auth UID -> account -> Person -> permission. UI gates are UX only.
2. Least privilege at login - Authenticated != authorized. Default deny until permissions resolve.
3. Invitation secrets - Activation links must be unguessable, time-limited where practical, and single-use or rotated on reuse policy.
4. PII - Email on Auth and User Account must stay consistent with Person contact policy; changes audited.
5. Disabled accounts - Immediate denial of app data access; sessions should be treated as invalid for operational reads/writes.
6. Service accounts / admin SDKs - Server paths still write audit trails and respect org isolation.
7. Public registration - Unauthenticated writes only to approved public intake paths; never to admin collections.

5.11 Soft delete and account removal

- Prefer disabled / revoked over hard-deleting Auth users during normal ministry ops.
- Soft-deleted Person does not imply Auth deletion; disable the account and block org context.
- Hard deletion of Auth users is exceptional and must not orphan operational history (Person remains).

5.12 Audit events (Auth / account)

At minimum, audit:

- Invitation created / resent / canceled
- Account activated
- Account disabled / re-enabled / revoked
- Person ? account link changes

- Successful and failed privileged auth-adjacent admin actions
- Active organization context switches for multi-org users (when enabled)

5.13 Chapter completion criteria

- Auth UID -> User Account -> Person chain is mandatory for signed-in operations.
- Newcomers can be shepherded without accounts; ministers with system duties require accounts.
- Member default is invite-after-approval activation, not silent auto-passwords.
- Account status is independent of ministry status.
- Active organization context scopes Person and permissions.
- Multi-org access is explicit; default deny across orgs.
- UI restrictions never replace Security Rules / trusted backend checks.

5.14 Next chapter

Chapter 6 defines the Authorization and Permission Engine: scopes, role templates, live updates, overrides, time-bound assignments, resolution order, Follow-Up defaults, and enforcement layers.

Chapter 6 - Authorization and Permission Engine

6.1 Purpose

This chapter defines how IEEC YA Connect decides whether a Person may view data or perform an action. The Permission Engine is a shared platform engine. Modules declare permissions; they do not invent private authorization systems.

Authentication (Chapter 5) answers who is signed in.

Authorization answers what that Person may do in a given scope.

6.2 Design principles

1. Default deny - absence of permission means deny.
2. Least privilege - grant only what the assignment requires.
3. Roles are templates, not final authority (ADR-002 / ADR-RBAC-001).
4. Live templates - template edits apply to current and future assignments (ADR-RBAC-002). Snapshotting permissions onto assignments is forbidden.
5. Default full-template grant - assigning a role grants all permissions currently on that template, evaluated inside the assignment scope. Admins do not pick permissions one-by-one at assignment time.
6. Overrides for exceptions - grant or deny on a Person without cloning a special role.
7. Explicit deny wins over grants.
8. Scope-aware - permissions never cross scope automatically (Chapter 3).
9. Time-aware when configured - optional start/end dates and active/inactive (ADR-RBAC-003).
10. Backend enforcement - React UI must mirror permissions but Security Rules / trusted functions enforce them.
11. Audited - permission-mutating operations write append-only history (ADR-006).
12. Technical authority != ministry authority - System Admin is not automatically Head Leader; Follow-Up Leader is not System Admin.

6.3 Permission scopes

Permissions are always evaluated inside a scope. Canonical scopes:

Scope | Meaning

platform | Platform-wide technical operations

parent_organization | Explicit parent oversight / standards

organization | Single church / branch tenant

ministry | e.g. Young Adult

team | e.g. Follow-Up Team

group | e.g. G5 or Bible Study subgroup

specific_record | One workflow/record instance when required

Scope chain example:

IEEC YA (organization) -> Young Adult (ministry) -> Follow-Up (team)

A Follow-Up Leader assignment at that team scope does not grant Worship or Media leadership.

6.4 Building blocks

6.4.1 Permission

A named capability key, preferably stable and namespaced:

```
follow_up.assignments.create  
follow_up.reports.review  
calendar.event.manage
```

Normalize singular/plural variants before implementation. Prefer plural resource segments (assignments, reports) as the canonical form.

6.4.2 Role template

Reusable named set of permissions (e.g. Follow-Up Leader). Has no authority until assigned within a scope.

6.4.3 Role assignment

Links:

- personId
- roleTemplateId
- organizationId
- scope type + scope ids (ministry/team/group as applicable)
- optional startAt / endAt
- active flag
- audit metadata

6.4.4 Permission override

Person-scoped exception:

- grant or deny
- permission key
- scope
- optional time bounds
- reason + audit metadata

6.4.5 Related access sources (not role titles alone)

- Team membership - baseline ops for members/ministers of a team
- Organizational position - Head Leader / Core Team defaults
- Oversight assignment - selected-team oversight permissions
- System role - technical admin capabilities
- Volunteer assignment - temporary scoped grants (prefer time-bound role assignment)

Backend evaluates resolved permissions, never "if title == Leader" string checks in isolation.

6.5 Default grant rule (approved)

When a role is assigned:

```
Person receives ALL permissions defined on that role template  
within the assignment's scope  
for as long as the assignment is effective
```

Examples:

Person: John

1. Follow-Up Leader @ IEEC -> YA -> Follow-Up
-> all Follow-Up Leader template permissions in that team scope
2. Bible Study Leader @ IEEC -> YA -> Bible Study
-> all Bible Study Leader template permissions in that team scope

If Admin later adds a permission to the Follow-Up Leader template, John's assignment receives it automatically (live templates). To withhold one permission from John only, add a deny override.

6.6 Time-bound assignments (approved)

Every role assignment supports:

- Optional start date
- Optional end date
- Active / inactive status

Expired or inactive assignments contribute zero permissions. Temporary Acting Team Leader, Conference Coordinator, and Event Volunteer Leader patterns rely on this instead of manual cleanup when possible.

Overrides may also be time-bound.

6.7 Follow-Up role template defaults

Detailed keys: docs/modules/follow-up-permission-catalog.md.

Role template | Default posture

Follow-Up Leader | Full Follow-Up management permission set within team scope

Follow-Up Assistant Leader | No management permissions by default; add explicitly to template or via overrides

Follow-Up Minister / team member | Assigned-work operations only (own newcomers: report, attendance, non-sensitive bio, recommend membership)

Assistant Leader title alone must not imply Leader authority. Basic assigned-work access comes from team membership / minister template, not from the Assistant title.

6.8 Permission resolution order

Evaluate for (personId, organizationId, requestedPermission, requestedScope):

0. System / platform restrictions (kill-switch, disabled account, archived person)
1. Explicit individual DENY override (effective in scope) -> DENY
2. Explicit individual GRANT override (effective in scope) -> ALLOW
3. Oversight assignment permissions (effective, matching scope)
4. Organizational-position permissions (effective, matching scope)
5. Team-role template permissions from effective assignments
6. Basic team-membership / minister baseline permissions
7. Default DENY

Notes:

- "Matching scope" includes the requested scope and correctly inherited parent scopes only where the permission definition and assignment say inheritance is allowed. Do not invent cross-team inheritance.
- When unioning role templates, a permission is present if any effective assignment grants it-unless a deny override applies.
- Parent-organization oversight never silently includes child operational permissions (ADR-007).

6.8.1 Traceability

For debugging and audit UX, the engine should be able to explain:

- which assignments/overrides contributed
- why deny won
- whether time bounds excluded an assignment

6.9 Enforcement layers

Layer | Duty

React UI | Show/hide routes, buttons, fields by effective permissions

Firestore Security Rules | Enforce the same decisions on reads/writes

Cloud Functions / Admin SDK paths | Re-check permissions for privileged workflows; write audit

Workflow engine | Gate transitions with required permissions + valid state

Hiding a button is not security. Every sensitive write path must fail closed without permission.

6.10 Admin configuration surfaces

Authorized admins (scoped) may:

1. Maintain role templates (permission sets)
2. Assign / end-date / deactivate role assignments
3. Create grant/deny overrides with reason
4. Manage oversight and organizational position assignments
5. View permission traces for a Person (support/admin)

Template edits are live; communicate impact to admins in UI copy where practical.

6.11 Audit requirements

Audit at least:

- Role template create/update/delete (soft)
- Role assignment create/update/deactivate
- Override create/update/deactivate
- Oversight / position assignment changes
- Privilege-sensitive workflow overrides (workflow.override)

Audit records are append-only and protected from normal client modification (ADR-006).

6.12 Multi-organization authorization

- Every assignment and override is organization-scoped (or parent/platform where explicitly modeled).
- Resolving permissions for org A must ignore org B assignments.
- Platform system roles are separate from organization ministry roles.
- Cross-org data access requires an explicit permissioned workflow, not shared role bleed.

6.13 Module contract

Each module must publish:

1. Permission catalog (stable keys + intent)
2. Default role template mappings
3. Which actions are UI-only vs security-critical
4. Record-level rules (own vs all, assigned-only, sensitive fields)

Follow-Up is the reference implementation of this contract.

6.14 Chapter completion criteria

- Roles are scoped live templates with full default grant and override exceptions.
- Time-bound assignments are first-class (ADR-RBAC-003).
- Deny overrides win; default deny otherwise.
- Scopes never auto-cross teams/orgs.
- Follow-Up Leader / Assistant / Minister defaults match the approved posture.
- UI + Firestore rules (+ trusted functions) enforce the same resolved permissions.
- Permission changes and template edits are audited.
- Evaluation uses resolved permissions, not role title string checks alone.

6.15 Next chapter

Chapter 7 will define the Dynamic Forms Engine (configurable registration and operational forms, field sensitivity, and validation), which Follow-Up registration and weekly reports consume.

Chapter 7 - Dynamic Forms Engine

7.1 Purpose

The Dynamic Forms Engine is the shared platform service for configurable data capture. Modules must reuse it for registration, operational reports, applications, schedules, and similar structured input instead of hard-coding one-off form schemas whenever practical.

7.2 Design principles

1. Configuration before hard-coding - field labels, order, required/optional/hidden, and options are admin-configurable within safeguards.
2. Published forms are versioned - a submission always retains the exact structure used at submit time.
3. Security is not configurable away - form config cannot grant unauthorized field visibility or bypass permissions.
4. Sensitivity-aware - fields may be classified (standard / sensitive / pastoral / internal-only).
5. Organization-scoped - form definitions belong to an organization (or parent/shared template published into orgs).
6. Module consumers, not owners - Follow-Up owns when to show a form; the Forms Engine owns definition, versioning, and submission shape.

7.3 Core concepts

Concept | Meaning

Form definition | Named form template (e.g. Public Newcomer Registration, Weekly Follow-Up Report)

Form version | Immutable published snapshot of fields + validation

Draft | Editable unpublished definition

Submission | Answers bound to formDefinitionId + formVersion

Field | Typed input with key, label, validation, visibility, sensitivity

7.4 Supported field types (baseline)

- Short text / long text
- Yes / no (boolean)
- Single-select / multi-select
- Date / time / datetime
- Number
- Phone / email (with normalization hooks where used for Person matching)
- File / photo reference (via File service)
- Conditional fields (show/require based on other answers)

Configurable per field: label, help text, order, required/optional/hidden/internal-only, default value, option lists, validation rules, sensitivity class, permission to view/edit answers.

7.5 Versioning rules

Draft -> Publish (creates immutable version N)

-> Later edits create Draft -> Publish as N+1

Rules:

- Submissions store formDefinitionId + formVersion (+ optional embedded field snapshot if needed for offline audit).
- Editing a live definition does not rewrite historical submissions.

- Admins may deprecate old versions; existing submissions remain readable with their original schema.
- Breaking key renames on new versions must not corrupt old dynamicResponses maps.

7.6 Submission model

Shared collections (planning names):

- formDefinitions/{formId}
- formVersions/{versionId} or subcollection under definition
- formSubmissions/{submissionId} when a standalone submission is needed

Module records may embed form binding instead of a separate submission doc when the module record *is* the submission (e.g. followUpReports with formDefinitionId, formVersion, dynamicResponses). Both patterns are allowed; choose one consistently per use case.

7.7 Public vs authenticated forms

Mode | Rules

Public | Unauthenticated write only to approved intake paths; no internal-only fields; rate-limit / abuse controls; duplicate detection after submit

Authenticated | Respect Chapter 6 permissions; field-level sensitivity enforced on read/write

Public registration must never expose internal notes, admin fields, or pastoral content (Follow-Up requirement).

7.8 Follow-Up consumers (first module)

Use | Form role

Public / internal newcomer registration | Configurable required/optional/hidden fields

Weekly follow-up report | Predefined + dynamic sections; attendance not a report field

Welcome schedule extras | Optional dynamic responses on schedule records

Membership recommendation narrative | May use form sections for structured summaries

Hard-coded MVP shells are acceptable only as temporary seeds that still store formDefinitionId / formVersion for forward compatibility.

7.9 Permissions

Examples (normalize with Chapter 6 catalog):

- forms.definition.view / forms.definition.manage
- forms.publish
- Module-specific submit/view permissions remain on the module (e.g. follow_up.reports.submit)

Managing form structure is separate from submitting answers.

7.10 Validation and workflow interaction

- Client validates for UX; trusted backend / rules enforce required fields for the published version.
- Workflow Engine may require a specific form version before a transition (e.g. recommendation cannot submit without completed sections).
- Configuration must not allow invalid workflows (ADR-005 / Ch. 2).

7.11 Soft delete and audit

- Soft-delete definitions/versions when retiring forms.
- Audit: publish, field sensitivity changes, deletion, and privileged submission edits.

7.12 Chapter completion criteria

- Shared form definitions with versioned publish.
- Submissions retain submit-time structure.
- Field types and conditional/sensitivity controls exist as platform capability.
- Public forms cannot expose internal fields.
- Follow-Up registration and weekly reports consume the engine (or seeded definitions).
- Form config cannot bypass RBAC.

7.13 Next chapter

Chapter 8 defines the Workflow Engine that drives approvals and state transitions that forms often feed.

Chapter 8 - Workflow Engine

8.1 Purpose

The Workflow Engine manages state machines, approvals, required steps, exceptions, and transition history for ministry processes. Modules declare workflow types and permissions; they must not invent isolated approval frameworks.

8.2 Design principles

1. Explicit current state on every workflow-managed record.
2. Transitions validate who, when, and required conditions.
3. Full transition history; never silent overwrite.
4. Configurable labels, steps, and deadlines within guardrails.
5. Critical system state identifiers stay stable even if labels change.
6. Authorized manual intervention / workflow.override is audited.
7. Workflows trigger notifications and tasks when configured (Chapter 11).
8. Backend enforcement - UI cannot be the only gate.

8.3 Core concepts

Concept | Meaning

Workflow type | Named process (e.g. newcomer_journey, membership_approval, weekly_report)

Workflow instance | One running process on a record (journeyId, recommendationId, ...)

State | Current machine state

Transition | Allowed move from state A -> B with guards

Approval template | Configurable multi-step approval path

Approval step | One approver role/person/position requirement

Override | Privileged forced transition with reason

8.4 Transition contract

Every transition records at minimum:

- Workflow type + record id
- Previous state -> new state
- Action name
- Actor (personId / system)
- Timestamp
- Reason / comments when required
- Permission source / override flag
- Related ids (assignment, approval step, form submission)

8.5 Approval templates

Support:

- One-step or multi-step
- Sequential (default for membership)
- Parallel where explicitly configured

- Return-for-correction, reject, cancel, skip (when allowed), expire

Step states: pending - approved - rejected - returned_for_correction - skipped - cancelled - expired

Approver actions: approve, reject, return, request info, delegate/abstain/cancel when policy allows. Reject/return require comment.

Guardrails (mandatory)

- No approval path without a final outcome definition
- No Person -> Member without an approval path
- No circular approval graphs
- No deadline without timezone
- No sensitive category without defined viewers

8.6 Follow-Up workflows (reference consumers)

The Workflow Engine must be able to express at least:

1. Registration -> duplicate review -> journey create
2. Assignment / reassignment
3. First contact + active follow-up loop
4. Weekly report lifecycle (draft -> submit -> review / return / lock)
5. Attendance correction (history-preserving; may be lightweight workflow)
6. Pause / unable_to_contact / inactive / decline / close / reopen
7. Pastoral escalation
8. Membership readiness -> recommendation -> configurable approval -> Member transition

Authoritative Follow-Up states and edges: docs/modules/follow-up-workflows-and-state-transitions.md.

8.7 Membership approval (canonical example)

Configurable template example:

Follow-Up Minister -> Follow-Up Leader -> Core Team -> Head Leader

Outcomes:

Outcome | Effect

Approved | Person status -> Member; journey completed; assignments ended; notifications/tasks

Returned | Stays in approval; revise and resubmit

Rejected | Journey returns to active/review-ready; does not auto-close

Member transition must be transactional / safely recoverable and must write Person status history (Chapter 4).

8.8 Automation

Workflows may enqueue:

- Reminders (first contact 48h default, report due Friday, etc.)
- Escalations on overdue states
- Welcome message tasks

Automation is configuration-driven and may be toggled per organization. Automation never bypasses permissions.

8.9 Permissions

- Step assignees act from approver assignment + underlying permissions
- workflow.override for authorized forced transitions
- Module permissions still gate starting actions (recommend, assign, close, ...)

Role title alone is insufficient (Chapter 6).

8.10 Data placement

Prefer:

- State fields on the domain record (journeyStatus, reportStatus, ...)
- workflowTransitions / approvalInstances collections for history and multi-step approvals
- Organization scope on every record

Do not bury unbounded history arrays inside hot documents.

8.11 Chapter completion criteria

- Shared transition + approval model reused by modules.
- Configurable multi-step approvals with return/reject/cancel.
- Guardrails prevent Member without approval and invalid graphs.
- Full transition history and override audit.
- Follow-Up journey/report/membership flows map cleanly onto the engine.
- Notifications/tasks can subscribe to transitions.

8.12 Next chapter

Chapter 9 defines the Ministry Calendar Engine that attendance and schedules depend on.

Chapter 9 - Ministry Calendar Engine

9.1 Purpose

Each organization has one shared operational calendar. Ministries, teams, and groups create events inside that calendar under permission and conflict rules. Modules (Follow-Up attendance, welcome schedules, events) link to calendar events; they do not invent separate calendars.

9.2 Design principles

1. One calendar per organization (ADR-003).
2. Events declare organizing scope (org / ministry / team / group) but live in the shared calendar.
3. Conflict policies are explicit: informational, warning, approval-required, or hard-block.
4. Parent events affect selected organizations only through declared scope and policy.
5. Parent events must not silently delete local events.
6. Attendance and schedules reference calendarEventId.
7. Timezones are first-class (organization default; event may override when needed).

9.3 Event model (baseline)

calendarEvents/{eventId} includes at least:

- organizationId
- Title, description
- startAt / endAt / timezone
- Organizing team/ministry/group ids as applicable
- eventScope, eventPriority, conflictPolicy
- Recurrence metadata (or link to parent recurring series)
- eventStatus (scheduled, cancelled, ...)
- Audit fields

Follow-Up Saturday program (approved MVP)

- Weekly Saturday 6:30 PM-9:30 PM (org timezone, e.g. America/New_York)
- Used as the attendance target for newcomers
- Unique attendance: personId + calendarEventId

9.4 Recurrence

- Support recurring series with materialised or queryable occurrences for operational weeks.
- Attendance links to the occurrence used for that Saturday, not only the series template.
- Cancelling one occurrence must not erase historical attendance already recorded.

9.5 Conflict rules

When creating/updating an event:

1. Detect overlaps in the shared org calendar per policy.
2. hard_block - reject or require override permission (calendar.conflict.override).
3. warning / approval_required - allow draft pending resolution.

4. Overrides and exceptions are audited.

Organization-reserved program times (Saturday YA program) may use hard-block against conflicting bookings when configured.

9.6 Parent-organization events

Per Chapter 3:

- Parent declares affected organizations.
- Declares informational vs warning vs approval-required vs hard-blocking.
- Conflict with local events notifies local owners and enters review - no silent delete.
- Parent calendar overrides are audited.
- Publishing a parent event does not grant parent users access to local Person pastoral data.

9.7 Attendance linkage

Calendar Engine provides events; attendance records are module/domain data:

- Follow-Up: newcomerAttendance with statuses attended | did_not_attend | unknown
- Separate from weekly reports
- Leaders may correct with history

Future modules (Members, Events) may attach other attendance types to the same calendar without forking calendars.

9.8 Welcome schedules

Welcome schedule rows may reference calendarEventId + assigned welcomer Person. Extra fields may use Dynamic Forms (Chapter 7).

9.9 Permissions

Examples:

- calendar.event.create
- calendar.event.manage
- calendar.conflict.override
- Module permissions for recording attendance remain on Follow-Up / other modules

9.10 Chapter completion criteria

- Single shared org calendar; no module-private calendars.
- Recurrence + timezone support sufficient for weekly Saturday program.
- Conflict policies and audited overrides.
- Parent events scoped, non-destructive, auditable.
- Follow-Up attendance uniquely keys on personId + calendarEventId.

9.11 Next chapter

Chapter 10 defines Chat and Collaboration, which remains independent of team membership and calendar attendance.

Chapter 10 - Chat and Collaboration

10.1 Purpose

The Chat and Collaboration Engine provides organization-owned messaging channels for ministry coordination. Chat supports Follow-Up and other teams but must never become a hidden permission system.

10.2 Design principles

1. Chat membership != team membership (ADR-004).
2. Joining a channel does not grant team module permissions or protected ministry-record access.
3. Teams may have multiple channels.
4. Chat data is organization-scoped and isolated.
5. Parent channels (if any) do not grant local org operational data access.
6. Soft delete / moderation actions are audited.
7. UI membership lists are not security for Person records.

10.3 Core entities

Entity | Purpose

chatChannels | Channel metadata (org, related team optional, type, status)

chatMemberships | Who may participate in the channel

chatMessages | Message body, sender, timestamps, soft-delete

Optional threads / attachments | Via File service references

Messages must not store unbounded nested replies inside a single document when growth is expected - use collections.

10.4 Channel types (baseline)

- Team operational channel - default Follow-Up coordination
- Case / newcomer channel (optional later) - selected participants around one journey
- Cross-team invite channel - members from other teams without granting Follow-Up record access
- Parent coordination channel - selected participants across orgs; local data remains local

10.5 Membership rules

- Add/remove members with follow_up.chat.manage / chat.manage_members (or platform chat manage permissions) inside scope.
- Membership changes are audited.
- Removing team membership does not automatically remove chat membership unless policy says so (and vice versa).
- External-team participants see chat content only - not unassigned queues, sensitive bios, or others' reports unless separately permitted.

10.6 Follow-Up usage

Follow-Up Leader template may include chat create/manage by default. Assistant Leader does not get management by default (Chapter 6 / permission catalog). Ministers may participate when added to the channel.

Chat is not a substitute for:

- Weekly reports
- Attendance records
- Formal escalations
- Membership approvals

10.7 Security

- Organization isolation enforced in Security Rules.
- Message read/write requires channel membership (plus account active).
- Sensitive Person fields must not be mirrored into chat as a bypass; pastoral content belongs in bio/escalation with permissions.
- Search/export of chat is privileged and audited.

10.8 Moderation and retention

- Soft-delete messages; retain for authorized audit/history.
- Channel archive preserves history.
- Retention policies may be configured later; default is keep for ministry continuity.

10.9 Chapter completion criteria

- Channels and memberships independent of team membership.
- Multi-channel per team supported.
- Org isolation; parent channels do not leak local records.
- Follow-Up can create/manage channels under permissions without implying module access for guests.
- Membership and moderation actions audited.

10.10 Next chapter

Chapter 11 defines Notifications and Tasks that chat, workflows, and deadlines emit.

Chapter 11 - Notifications and Tasks

11.1 Purpose

Notifications and Tasks are shared platform services. Modules request them; they do not each build private reminder systems. The engine distinguishes informational notices from required actions, and tracks delivery, read state, due dates, and escalation.

11.2 Design principles

1. Organization-scoped delivery.
2. Separate notification (inform) from task (actionable work item).
3. Channel abstraction: in-app (web + mobile), push (especially mobile), email; SMS/WhatsApp later.
4. Idempotent triggers where possible (avoid duplicate storms).
5. User notification preferences may mute channels but must not disable critical security notices without admin policy.
6. Tasks have owners, due dates, status, and optional escalation.
7. Deep links go to permitted screens only - permission still enforced on arrival.

11.3 Notifications

Baseline fields

- organizationId, recipient personId / account
- Type / template key
- Title, body, data payload
- Channel(s)
- Related entity refs (journeyId, reportId, ...)
- createdAt, deliveredAt, readAt
- Status: pending - sent - failed - read - dismissed

Typical Follow-Up triggers

- New registration / unassigned queue
- Assignment created / reassigned
- First-contact deadline approaching / overdue (default 48h)
- Weekly report due / late / returned
- Membership recommendation pending approval
- Welcome message send result
- Escalation opened
- Journey pause review date

11.4 Tasks

Tasks represent required work:

Example | Owner | Due

First contact newcomer | Primary assignee | +48h default

Submit weekly report | Primary assignee | Friday due

Review membership recommendation | Approver step | Template SLA

Resolve duplicate review | Authorized reviewer | Configurable
Correct returned report | Report author | Edit window

Task statuses: open - in_progress - completed - cancelled - expired - escalated

Completing the underlying workflow action should auto-complete the linked task when configured.

11.5 Templates and configuration

Admin-configurable:

- Template copy per event type
- Channels enabled
- Reminder offsets
- Escalation paths
- Welcome message on/off (Follow-Up default: on)

Guardrail: no deadline automation without timezone (Chapter 8).

11.6 Delivery architecture (planning)

Baseline path:

1. Workflow / module writes notifications / tasks docs.
2. Cloud Functions (when enabled) fan out email and mobile/web push.
3. In-app inbox on web and mobile reads Firestore notifications for the Person.

Failures retry with backoff; permanent failures mark failed and may create admin-visible alerts.

11.7 Permissions and privacy

- Recipients see their own notifications/tasks.
- Leaders may see team task boards only with permission.
- Notification bodies must minimize sensitive pastoral content; prefer "new escalation needs review" over pasting bio text into email.
- Parent org notifications never include restricted local pastoral payloads by default.

11.8 Chapter completion criteria

- Shared notification + task models used by Follow-Up.
- Informational vs actionable distinction clear.
- Due dates, read/delivery tracking, and escalation supported.
- Config templates for Follow-Up deadlines and welcome message.
- Channel expansion (SMS/WhatsApp) possible without redesigning modules.

11.9 Next chapter

Chapter 12 defines Audit and History requirements that notifications, workflows, and all engines must satisfy.

Chapter 12 - Audit and History

12.1 Purpose

The Audit Engine records important actions as append-only history so ministry operations remain accountable. Soft delete is the default for operational records; audit explains who changed what and why.

12.2 Design principles

1. Soft delete by default; hard delete is exceptional and separately governed.
2. Audit records are append-only and protected from normal client modification.
3. Prefer writing audit from trusted backend / controlled paths.
4. Capture before/after or explicit action semantics for sensitive changes.
5. Include actor, permission context, timestamp, organization scope.
6. History must survive Person status changes and journey closure.
7. React UI restrictions never replace backend enforcement or audit.

12.3 What must be audited (minimum)

Domain | Examples

People | Status changes, merges, soft-delete/restore, sensitive profile edits

Auth / accounts | Invite, activate, disable, revoke, person?account link changes

Permissions | Role template edits, assignments, overrides, oversight changes

Follow-Up | Registration decisions, assignments/reassigns, report submit/edit/lock, attendance create/correct, bio add/soft-delete, journey transitions

Membership | Recommendation submit, approval steps, Member transition

Calendar | Event create/update/cancel, conflict overrides, parent publish effects

Chat | Channel membership changes, moderation deletes

Workflow | Every transition; especially workflow.override

Parent oversight | Access to approved aggregates / shared reports

Files | Upload/delete of sensitive attachments

12.4 Audit record shape (baseline)

auditLogs/{auditId}:

```
{
  organizationId: string
  actorPersonId: string | null
  actorAuthUid: string | null
  actorType: 'user' | 'system' | 'automation'
  action: string
  entityType: string
  entityId: string
  moduleKey: string | null
  previousValue: unknown | null
  newValue: unknown | null
  reason: string | null
  permissionKeys: string[] | null
  correlationId: string | null
  createdAt: Timestamp
}
```

Do not rely on mutable "last audit" fields alone.

12.5 Soft delete pattern

Operational docs use:

- recordStatus / deletedAt / deletedByPersonId
- Queries default to non-deleted
- Restore is a privileged, audited action

Attendance corrections and report edits keep prior versions or correction history rather than inventing a clean rewrite.

12.6 Retention and access

- Audit access is highly privileged (audit.view or system-admin scoped).
- Pastoral content inside audit payloads should be minimized; store refs + field diffs when possible.
- Retention follows organization/legal policy; platform default is long retention for ministry accountability.
- Export for incidents is admin-only and audited.

12.7 Correlation

Use correlationId (or workflow transition id) to group multi-doc transactions (e.g. Member approval updating Person + journey + assignments + notifications).

12.8 Chapter completion criteria

- Append-only org-scoped audit log exists.
- Required action classes above are covered for Follow-Up MVP paths.
- Soft delete + restore semantics documented and used.
- Client cannot freely rewrite audit history.
- Overrides and permission changes always audited.

12.9 Next chapter

Chapter 13 defines Data and Engineering Standards for Firestore modeling, security layers, testing, and AI/implementation conventions.

Chapter 13 - Data and Engineering Standards

13.1 Purpose

This chapter sets engineering standards so modules stay consistent, secure, and maintainable across web and mobile. AI coding assistants and human developers must follow these rules unless an ADR changes them.

13.2 Engines vs modules

Kind | Responsibility | Location (suggested)

Engine | Reusable platform capability | engines/*

Module | Business ministry functionality | modules/follow-up, etc.

Modules must reuse People, AuthZ, Forms, Workflow, Calendar, Chat, Notifications/Tasks, Audit, and Configuration engines. Duplicating person stores, calendars, chats, workflows, or form systems is non-compliant (Chapter 2 completion criteria).

13.3 Firestore data standards

1. Organization scope - every org-owned operational doc includes organizationId.
2. Authoritative scope - never trust UI path alone; Security Rules verify scope from records.
3. Top-level collections preferred for operational entities with organizationId (Follow-Up model).
4. Person is lean - journeys, reports, attendance, bio, messages live in separate collections.
5. No huge growing arrays inside hot documents for messages/reports/attendance.
6. Soft delete by default (ADR-006).
7. Timestamps - createdAt / updatedAt; actor ids on mutating fields where relevant.
8. Normalized contact fields for search/duplicate detection.
9. Idempotent unique keys where required (e.g. attendance personId + calendarEventId).
10. Indexes planned with queries; document composite indexes in planning/README as they are added.

13.4 Identity and security standards

Auth UID -> userAccounts -> Person -> effective permissions -> allow/deny

- Default deny; explicit deny wins (Chapter 6).
- UI hide != security.
- Enforcement layers (Chapter 2): React UI -> application/service validation -> Firestore Security Rules -> Cloud Functions/trusted backend -> automated allow/deny tests.
- Public write surfaces are allowlisted and minimal.
- Secrets never committed; use env/emulator configs.

13.5 Permission key standards

- Namespaced keys: domain.resource.action
- Prefer plural resources: follow_up.assignments.create, follow_up.reports.review
- Normalize legacy singular/plural aliases before coding freezes the catalog
- Evaluate resolved permissions, not role title strings

13.6 Configuration standards

- Deadlines, labels, approval templates, reminder offsets, welcome toggles are configuration.
- Configuration must not disable security guardrails or invent illegal Member transitions.
- Every deadline has a timezone.
- Seed org defaults from docs/modules/follow-up-config-defaults.md for Follow-Up.

13.7 API / function standards

- Privileged multi-doc transactions (Member approval, merge Person, invite activation) prefer Cloud Functions / callable trusted paths.
- Functions re-check permissions and write audit.
- Client may do direct Firestore access only where rules fully encode the policy.

13.8 Client standards (web + mobile)

- Product ships two clients: Web and Mobile. Neither is a permanent "phase 2 only" surface.
- Parity rule: Follow-Up minister operations (assigned queue, weekly report, Saturday attendance, bio, basic newcomer profile) must work on mobile and web. Heavy admin/config may start web-primary.
- Shared domain contracts (types, permission keys, workflow states) must not diverge per client.
- Permission-gated navigation and actions on every client.
- Engines expose shared services; each client composes UI only.
- Mobile UX is touch-first; web may optimize denser leader dashboards.
- Push notifications are expected for mobile task/reminder channels (Chapter 11).
- Do not redesign architecture for one client's convenience.

13.8.1 How iOS and Android installables are generated

The mobile app is a real native binary, not "save the website as an app." Builds produce:

Platform | Artifact | Typical use

Android | .apk | Internal/test install / sideload

Android | .aab (Android App Bundle) | Google Play Store upload (preferred for store)

iOS | .ipa | TestFlight / App Store (requires Apple Developer Program)

Settled stack (aligned with React web): Expo (React Native) + EAS Build

```
eas build --platform android -> .apk and/or .aab
eas build --platform ios    -> .ipa
eas submit                  -> Play Console / App Store Connect (optional)
```

Local/dev alternatives still exist (npx expo run:android, Xcode archive, Android Studio), but CI/cloud builds via EAS are the standard release path so the team does not depend on one person's laptop.

Flutter is not part of this baseline. Do not introduce a Flutter client.

Accounts and signing (required either way):

1. Google Play Console developer account - signing key / Play App Signing for Android store releases
2. Apple Developer Program - certificates, provisioning profiles, bundle id for IPA / TestFlight / App Store
3. Store listing, privacy policy, and Firebase config (google-services / GoogleService-Info.plist) wired per environment (dev/staging/prod)

Not how we ship mobile: wrapping the Vite web URL in a WebView-only shell as the long-term app. Web remains a first-class browser client; mobile is a native client on the same Firebase backend.

MVP distribution path: internal testers via Firebase App Distribution and/or TestFlight + internal Play track, then public store when ready.

13.9 Testing standards

Minimum before calling a slice done:

- Unit tests for permission resolution and workflow guards where logic is pure.
- Rules tests (emulator) for allow and deny paths on Follow-Up collections.
- Smoke tests for public registration and authenticated report/attendance happy paths on web and mobile.
- At least one successful Android build artifact (apk/aab) and one iOS build path documented before calling mobile "shippable" (TestFlight optional until Apple account is ready).

13.10 AI / human implementation protocol

1. Read docs/SOURCE_OF_TRUTH.md and handbook chapters.
2. Read ADRs; do not reopen approved decisions.
3. Classify change as Engine vs Module.
4. Follow docs/AI_CODING_HANDOFF_PROMPT.md for coding sessions.
5. If docs are silent, ask the human - do not invent product policy.
6. Planning agents do not ship application code; coding agents do not rewrite handbook policy.

13.11 Definition of architecture-compliant module

A module is compliant only when it:

- Reuses shared engines
- Declares permission catalog + default role mappings
- Protects sensitive data
- Preserves history / soft delete
- Writes required audit events
- Avoids duplicate person/calendar/chat/workflow/form infrastructure
- Stays organization-scoped under ADR-007

13.12 MVP coding freeze note

For the first Follow-Up coding milestone, Chapters 1-13 drafts + Follow-Up module pack + ADRs are sufficient to implement Phase A (platform foundation) and Phase B (Follow-Up MVP) for web and mobile in the handoff prompt. Parent-org product depth, SMS/WhatsApp, and other ministry modules remain out of scope until specified.

13.13 Chapter completion criteria

- Engine/module boundary and Firestore standards are explicit.
 - Web + mobile clients are both first-class with shared backend contracts.
 - Security enforcement layers are mandatory on every client.
 - Permission naming and config guardrails documented.
 - Testing and AI implementation protocol defined.
 - Compliance checklist exists for future modules.
-

Handbook draft set status: Chapters 1-3 (PDF v0.3) + Chapters 4-13 (repo drafts) form the planning baseline for human freeze before coding.